이 페이지

1. Greedy란
2. Selecting Breakpoints
3. Minimum Spanning Tree
4. Kruskal
5. Cut Property — MST 정확성의 엔진
6. Prim
7. Union-Find (disjoint sets)
8. Interval Scheduling
9. Interval Partitioning
10. Huffman Encoding
11. Clustering of Maximum Spacing
12. 연습문제

그리디 (greedy) 알고리즘

2026-09-28 (월) · 09-30 (수) · 10-05 (월) · 슬라이드 Greedy Algorithms (p.1–33, 녹취 없음, 교수 marked PDF)

Greedy란

해를 조각조각 쌓되, **매 단계에서 가장 확실하고 즉각적인 이득**을 주는 조각을 고른다. 지금 상황에서 제일 좋아 보이는 선택 = 직관에 기대는 방식 (예: 지금 이익이 큰 회사의 주식에 투자).

- **쉬운 쪽** — 설계. DP 같은 다른 틀보다 알고리즘을 떠올리기 쉬움
- **어려운 쪽** — 그 선택이 정말 **최적임을 증명**하는 것 + 수행 시간 분석

이 단원의 증명 도구: induction (Breakpoints), **cut property**와 교환 논증 (MST), 교환 논증 (interval scheduling, Huffman)

Selecting Breakpoints

- 상황: 고정된 경로를 달리는 여행 (전기차 LA → New York), 완충하면 최대 C km, 출발 시 완충
- 목표: 충전소에서 멈추는 횟수 **최소**

- **Greedy: 갈 수 있을 때까지 최대한 멀리** 간 뒤 충전 — 후보 4개 중 첫 충전소를 고르면 거기서 500 km, 마지막 충전소를 고르면 더 앞선 위치에서 500 km 출발 → 늦게 충전할수록 유리
- 정당성 증명(귀납법). 출발점에서 목적지까지의 거리를 L, 한 번 충전으로 이동할 수 있는 최대 거리를 C라고 하자. 먼저 $L \leq C$이면 출발 시 충전 없이 목적지에 도달하므로 greedy는 최적이다.

  이제 $L > C$이고 목적지에 도달할 수 있는 경로가 존재한다고 하자. 최적해의 충전 위치를 $(x_1, \ldots, x_m)$, greedy의 충전 위치를 $(g_1, g_2, \ldots)$라고 하자. Greedy는 출발점에서 도달 가능한 가장 먼 충전소를 선택하므로 $g_1 \geq x_1$이다.

  귀납적으로 $g_k \geq x_k$라고 가정하고 greedy가 아직 목적지에 도달하지 않았다고 하자. 최적해의 다음 충전 위치 x_{k+1}이 존재하면 $x_{k+1} \leq x_k + C \leq g_k + C$이다. 따라서 greedy가 $g_k + C$ 이내에서 선택하는 가장 먼 충전소 g_{k+1}은 $g_{k+1} \geq x_{k+1}$을 만족한다. 반대로 $L \leq g_k + C$이면 greedy는 다음 충전 없이 목적지에 도달하므로 이후의 충전 위치를 정의할 필요가 없다.

  최적해가 m번 충전하여 목적지에 도달한다고 하자. Greedy가 그보다 먼저 도착하면 이미 충전 횟수가 m회 이하이므로 충분하다. 그렇지 않으면 귀납적으로 $g_m \geq x_m$이고, $L \leq x_m + C \leq g_m + C$이므로 greedy도 늦어도 m번째 충전 후 목적지에 도달한다. 따라서 greedy의 충전 횟수는 최적해의 충전 횟수 이하이다. 최적해가 최소 충전 횟수를 갖는다는 정의에 의해 greedy 역시 최적이다.

  또한 현재 위치 x에서 $L > x + C$이고, x보다 앞에 있으면서 $x + C$ 이하인 충전소가 없다면 어떤 경로로도 다음 충전을 수행할 수 없다. 따라서 이 경우 no solution을 반환하는 것도 정당하다.

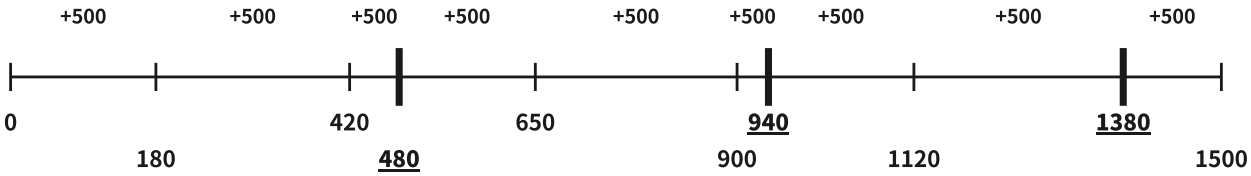
SelectBreakpoints(B): 충전소를 $0 = b_0 \leq b_1 \leq \cdots \leq b_n = L$ 로 정렬, $S = \{0\},\ x = 0$

- $x \neq b_n$인 동안: $b_p \leq x + C$를 만족하는 **가장 큰** p를 잡음
- $b_p = x$이면 제자리 → 더 못 감 → **return "no solution"**
- 아니면 $x = b_p$, $S = S \cup \{p\}$
- 목적지 $b_n = L$은 실제 충전소가 아닌 종료 지점이다 — L ≤ x + C이면 목적지에 도달해 성공적으로 종료하고, 목적지 도착은 충전 횟수에 세지 않는다. 현재 위치 x, 목적지 L, 1회 충전 최대 거리 C.

  L > x + C이고 x보다 앞에 있으면서 x + C 이하에 위치한 충전소가 없다면 no solution을 반환한다. 도달 불가능한 위치에 충전소가 남아 있어도 판정은 달라지지 않는다

시간: 정렬돼 있으면 $O(n)$, 정렬이 필요하면 $O(n \log n)$ 추가

선형 시간은 포인터를 앞으로만 이동시켜 x + C 이내의 가장 먼 충전소를 찾는 구현을 전제로 한다. 포인터가 뒤로 돌아가지 않으므로 전체 충전소를 합쳐 O(n)번만 검사한다. 매 단계마다 처음부터 다시 탐색하는 구현은 이 시간복잡도를 보장하지 않는다



충전 3번: 480, 940, 1380 — 매번 갈 수 있는 가장 먼 충전소

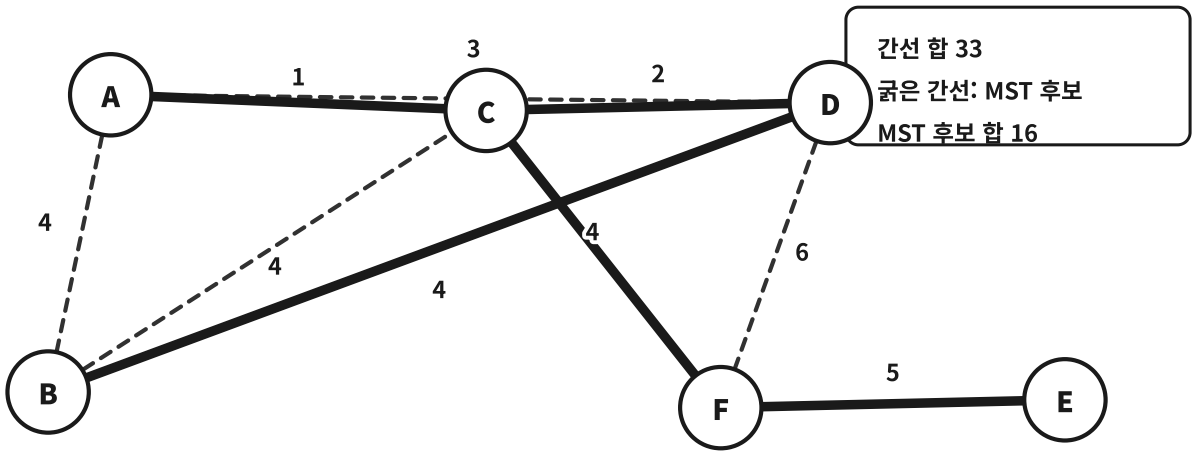
손계산 1 — 충전소 0, 180, 420, 480, 650, 900, 940, 1120, 1380, 1500 ($L = 1500, C = 500$).
슬라이드에 수치가 없어 노트가 만든 예. 선택: 0 → 480 → 940 → 1380 → 1500, 충전 3번. 2번으로는 불가능하다 — 충전 위치가 두 곳 x, y뿐이면 첫 충전소는 닿을 수 있는 가장 먼 곳이 480이라 x ≤ 480이고, 목적지 1500에 닿으려면 y ≥ 1000이어야 하지만, y ≤ x + 500 ≤ 980이라 모순이다 → 3번이 최소. no solution 예: 충전소 0, 300, 860, 1000, $C = 500$ → 300에서 800까지밖에 못 감 → $b_p = x$로 종료

Minimum Spanning Tree

문제: 가장 싼 연결 네트워크

- 상황: 캠퍼스 건물 6개(A-F)를 광케이블로 연결, 연결마다 연 유지비 (백만 원). 일부 연결을 끊되 건물은 모두 연결된 채로 → **총 유지비 최소**
- cycle이 남아 있으면 그 안의 간선 하나를 지워도 연결 유지 (가중치 양수라 더 싸짐) → **최적해에는 cycle이 없음** → connected + acyclic = **tree**

MST	비용 최소의 **spanning tree** (모든 정점을 잇는 tree)
조건	간선은 반드시 주어진 그래프의 것, 간선 수 $\|V\| - 1$, 이 예제에서는 간선 가중치가 모두 양수라고 가정한다. 일반적인 MST에서는 가중치가 음수나 0이어도 MST·Kruskal·Prim·cut property가 적용된다
입력 가정	입력은 가중치 있는 연결 무방향 그래프라고 가정. 비연결이면 MST는 없고 Kruskal은 최소 신장 포리스트를 구성, Prim은 각 연결요소에서 재시작해야 전체 포리스트를 얻는다
참고	같은 가중치가 있으면 MST가 여러 개일 수 있으나 합은 같음
목표	$weight(T) = \sum_{e \in E'} w_e$를 최소화하는 $T = (V, E'),\ E' \subseteq E$



MST 후보 합 16 · B–D 대신 A–B, B–C 중 하나를 선택해도 16

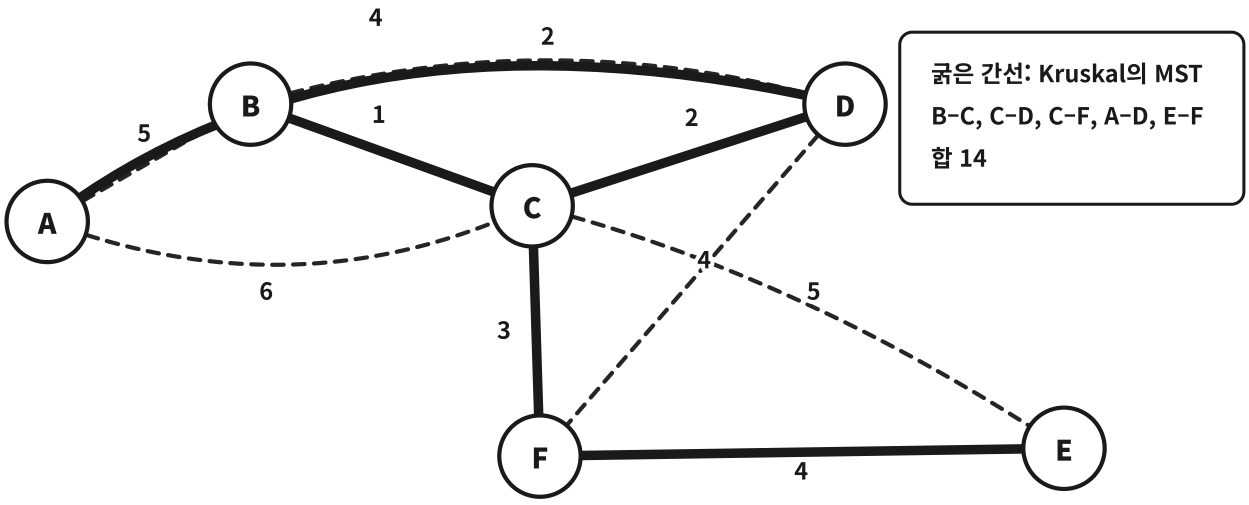
슬라이드 3 그래프 — 이 그래프 기준 Kruskal 결과 합 16 (연습문제 2 참고)

- 슬라이드 그래프: 간선 9개의 합 33 → 오른쪽 tree 합 16
- 응용: 캠퍼스 건물 23개 + 네트워크 서버, 광케이블 비용 $\propto$ 길이 → 완전 그래프에서 MST. Euclidean 거리 vs geodesic 거리 → **거리 정의만 바꾸면 같은 문제**. PCB 회로 기판도 신호 지연 $\propto$ 길이 → 같은 모델

Kruskal

빈 그래프에서 시작, 간선을 **가벼운 순**으로 보며 **cycle을 만들지 않으면 추가**.

- 간선 1개, 2개는 cycle이 불가능 → 처음 두 개는 무조건 추가, 세 번째부터 cycle 검사
- 중간 상태 X = 여러 tree의 모임 (**forest**, 부분해) → 다음 간선 = 서로 다른 두 tree를 잇는 가장 가벼운 간선



Kruskal이 고른 MST: B–C, C–D, C–F, A–D, E–F · 합 14

슬라이드 5 그래프 — Kruskal이 고른 MST (합 14). 이후 모든 MST·union-find 예제의 기준 그래프

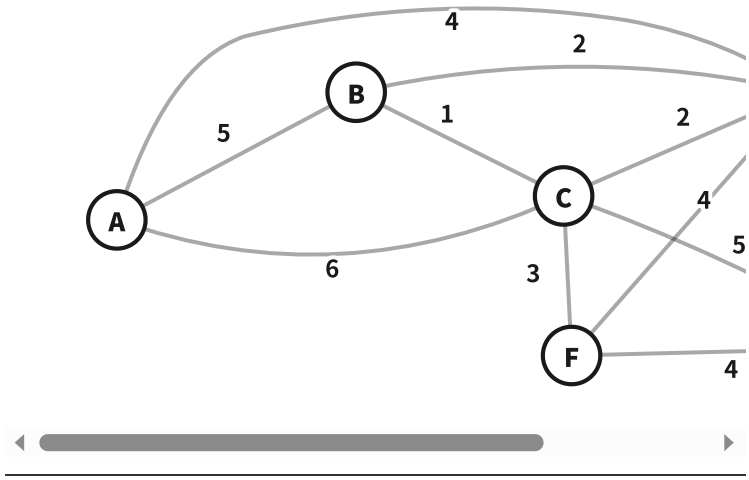
손계산 2: Kruskal (슬라이드 5 그래프)

순서	간선	가중치	find(u), find(v)	판정	남은 집합	누적 weight
1	B–C	1	B, C	**추가**	5	1
2	C–D	2	C, D	**추가**	4	3
3	B–D	2	C, C	버림 (같은 집합)	4	3
4	C–F	3	C, F	**추가**	3	6
5	A–D	4	A, C	**추가**	2	10
6	D–F	4	C, C	버림	2	10
7	E–F	4	E, C	**추가**	1	14
8-10	A–B 5, C–E 5, A–C 6	모두 같은 집합		버림	1	14

Kruskal 단계별 시뮬레이션

간선을 가중치 오름차순으로 검사하고, 서로 다른 집합이면 추가한다.

이전 | 다음 | 초기화 | 자동 재생



굵은 실선: MST에 추가 --- 점선: 버림

흐린 선: 아직 검사 전 가장 굵은 선: 현재 검사 간선

현재 단계의 상태

검사 중인 간선

—

find(u)

—

find(v)

—

판정

시작 전

남은 집합 수

6

누적 weight

0

남은 집합

대표 A {A} | **대표 B** {B} | **대표 C** {C}

대표 D {D} | **대표 E** {E} | **대표 F** {F}

집합 목록의 대표는 사전순 최소 정점이다. find(u)와 find(v)는 Union-Find 트리의 루트를 표시한다.

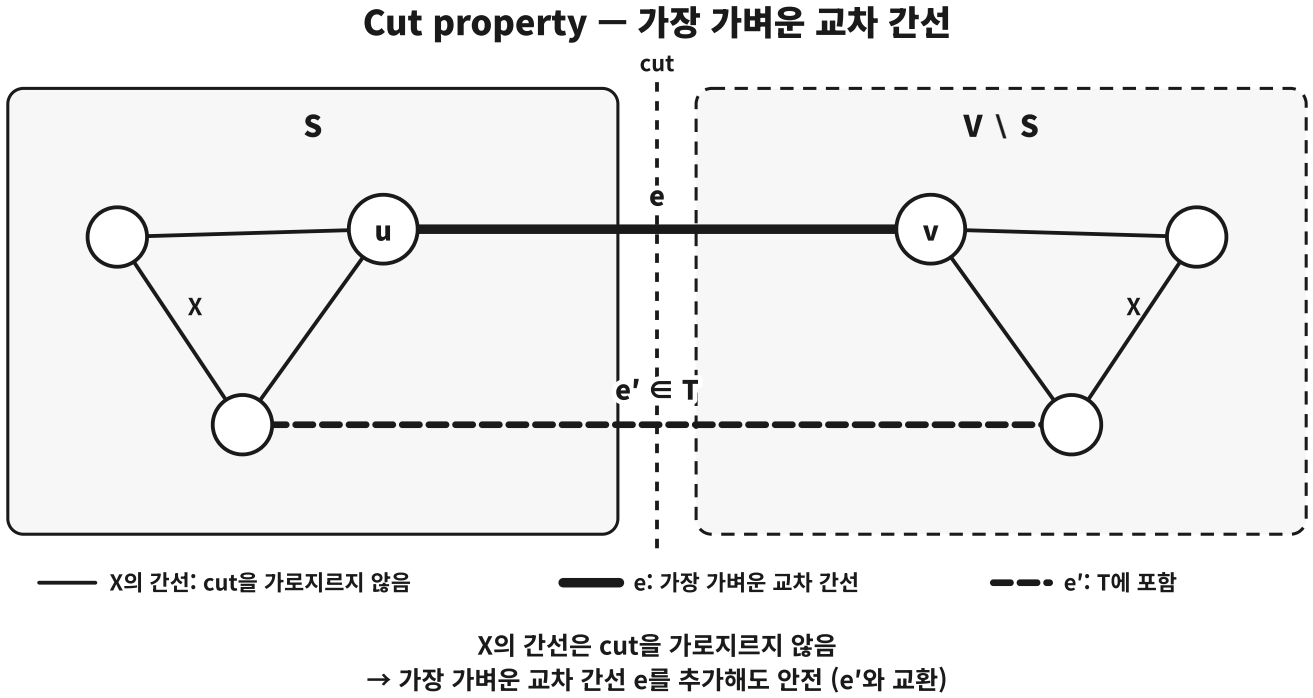
선택된 MST 간선 목록

없음

- 노드 6개짜리 MST는 간선 5개 → 5개 모이면 완성, 나머지 3개는 전부 버려짐. 집합 수 6 → 1
- **MST가 2개인 이유**: B–D 2를 쓰는 MST (B–C 1, B–D 2, C–F 3, A–D 4, E–F 4) 도 합 14. A는 A–D 4, E는 E–F 4가 유일한 최소 연결 → 고정. B, C, D, F 사이에서 B–C 1, C–F 3 고정, 2짜리만 C–D 또는 B–D → 2가지 (완전탐색 $\binom{10}{5} = 252$가지로 확인 가능)

Cut Property — MST 정확성의 엔진

$X \subseteq E$가 어떤 MST의 부분집합이고, $S \subseteq V$를 가로지르는 X의 간선이 없다고 하자. S와 $V \setminus S$ 사이를 가로지르는 **가장 가벼운 간선** e에 대해 $X \cup \{e\}$도 어떤 MST의 부분집합이다.



증명: 교환 논증 (exchange argument)

① $e = (u, v) \notin T$라 가정 (T = X를 포함하는 MST)

② T 안에서 u–v 경로는 cut을 가로지르는 간선 e'를 반드시 포함

③ $T' = T - e' + e$: e가 최소이므로 $w_e \leq w_{e'} \rightarrow \text{cost}(T') \leq \text{cost}(T) \rightarrow T'$도 MST

④ $e' \notin X$이므로 T'은 $X \cup \{e\}$를 포함 ∎

- **Kruskal에 적용 — 정당성은 두 경우로 나뉜다**: 간선 $e = (u, v)$를 가벼운 순으로 검사할 때
  - **서로 다른 연결요소** (검사 조건 `find(u) ≠ find(v)`): $e = (u, v)$를 추가하면 두 tree가 하나로 합쳐진다. S = u가 속한 tree로 두면 X의 간선은 tree 안에만 있어 cut을 안 가로지르고, 이보다 가벼운 간선은 이미 검사했을 때 가로지르는 것이었다면 추가됐을 것 → e가 교차 간선 중 최소 → cut property로 안전

  - **같은 연결요소** (검사 조건 `find(u) = find(v)`): 두 끝점 사이에 이미 경로가 있어 e를 추가하면 cycle이 되므로 버린다

  즉 `find(u) ≠ find(v)` 검사가 이 두 경우를 가르는 검사다

- **주의**: Kruskal이 B–D 2 (슬라이드 5) 를 버린 건 B, C, D가 이미 연결돼 cycle이 되기 때문이지 안전하지 않아서가 아님.
- cut ({B, C}, 나머지)에서 B–D 2는 C–D 2와 공동 최소 교차 간선이라 cut property상 안전. 실제로 B–D를 쓰는 MST (B–C 1, B–D 2, C–F 3, A–D 4, E–F 4) 도 합 14로 존재 (위 손계산 2)
- $X = \varnothing$이어도 성립 → 맨 처음부터 적용 가능, 매 반복마다 다시 적용
- 손계산 2 그래프에서 Kruskal 각 단계의 cut:
  - B–C (1): $S = \{B\}$ — 교차 B–C 1, B–D 2, A–B 5 → 최소 1
  - C–D (2): $S = \{B, C\}$ — 교차 C–D 2, B–D 2, C–F 3, A–B 5, C–E 5, A–C 6 → 최소 2 (B–D 2와 동점 → 어느 쪽이든 MST)
  - C–F (3): $S = \{B, C, D\}$ — 교차 C–F 3, A–D 4, D–F 4, A–B 5, C–E 5, A–C 6 → 최소 3
  - A–D (4): $S = \{A\}$ — 교차 A–D 4, A–B 5, A–C 6 → 최소 4
  - E–F (4): $S = \{E\}$ — 교차 E–F 4, C–E 5 → 최소 4

Kruskal 구현과 시간

- makeset $|V|$번, find $2|E|$번 (간선마다 두 끝점), union $|V| - 1$번
- find(u) ≠ find(v) → 다른 집합 → 추가 + union (집합 수 1 감소); 같으면 추가 시 cycle이라 버림 (근거는 위 Kruskal에 적용의 두 경우)
- 순진 구현 (연결 성분을 매번 BFS): $O(|E|(|V| + |E|))$ → 완전 그래프 $|E| \approx n^2$에서 n^4 규모 → 너무 느림
- union-find 사용: 총 $O(|E| \log |V|)$ — 정렬 $O(|E| \log |E|)$, find $2|E|$번 $\times$ $O(\log |V|)$. $\log |E| \leq 2 \log |V|$라서 $O(|E| \log |E|) = O(|E| \log |V|)$

Prim

시작 노드 u_0 하나에서 출발, S = 지금까지 만든 subtree의 노드들. 매 단계 S**와** $V \setminus S$ **사이의 가장 가벼운 간선**으로 확장.

- **정당성**: cut $(S, V \setminus S)$에 cut property를 적용한 것이다. 지금까지 고른 간선은 모두 S 안에 있어 이 cut을 가로지르지 않으므로, S와 $V \setminus S$ 사이의 가장 가벼운 간선을 추가해도 안전하다. 매 단계 이 간선으로 확장하므로 Prim이 고른 간선 집합은 항상 어떤 MST의 부분집합이다
- 상태 변수 (노드마다 하나씩):
  - $\mathrm{cost}(z)$ = S와 z를 잇는 최소 간선 가중치
  - $\mathrm{prev}(z)$ = 그 간선의 S쪽 끝점
- deletemin으로 cost 최소인 v를 뽑아 S에 추가 → v의 이웃 z마다 아래 검사 조건을 만족하면 갱신 + decreasekey
  - 검사 조건: $\mathrm{cost}(z) > w(v, z)$
- 시간: $O(|E| \log |V|)$ (binary heap)

슬라이드 의사코드에는 z가 아직 큐에 있는지 확인이 없음 → 이미 S에 들어간 z는 건드리면 안 되므로 실제 구현에서는 **큐에 남은** z**만 갱신**.

손계산 3: Prim (시작 A, 슬라이드 5 그래프)

칸 = cost(z)/prev(z), 빈 칸 = 이미 S에 들어간 노드

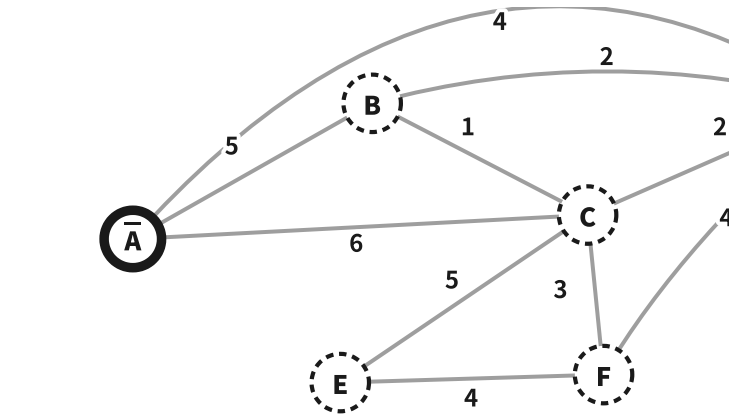
S	A	B	C	D	E	F
	0/nil	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
A		5/A	6/A	4/A	∞/nil	∞/nil
A, D		2/D	2/D		∞/nil	4/D
A, D, C		1/C			5/C	3/C
A, D, C, B					5/C	3/C
A, D, C, B, F					4/F	

단계 1 / 6 · 초기화

Prim 단계별 시뮬레이션

시작 정점 A에서 출발해 S 밖의 최소 cost(z) 정점을 선택한다.

이전 | 다음 | 초기화 | 자동 재생



선택된 MST 간선 이번 단계의 간선
아직 선택되지 않은 간선 점선 테두리 정점: S 밖

현재 단계의 상태

현재 집합 S

S = {A}

방금 뽑은 정점

A · 시작 정점 · cost(A)=0, prev(A)=nil

선택 정점의 cost

0

누적 합

0

단계 설명

S={A}. S 밖의 각 정점에 대해 현재 알려진 최소 연결 비용 cost와 이전 정점 prev를 기록한다. A와 직접 연결된 후보는 A–D(4), A–B(5), A–C(6).

시작 정점 A의 초기값은 cost(A)=0, prev(A)=nil이다.

cost / prev 표

선택된 MST 간선

1. 아직 선택된 간선 없음

정점 z	cost(z) / prev(z)
A S	
B	5 / A
C	6 / A
D	4 / A
E	∞ / nil
F	∞ / nil

칸은 cost(z) / prev(z)를 나타낸다. 빈 칸은 이미 S에 들어간 정점이다. 초기화 시 시작 정점 A는 cost(A)=0, prev(A)=nil로 설정된다.

선택된 간선의 가중치 합이 누적 합이다.

- 뽑는 순서 A → D(4) → C(2) → B(1) → F(3) → E(4), 합 14 — Kruskal 결과와 같은 tree
- A, D 단계에서 B와 C가 2로 동점 → B를 먼저 뽑으면 B–D 2를 쓰고 다음에 C가 1/B → **다른 MST** (합 14, Kruskal의 두 번째 MST)

비교

항목	Kruskal	Prim
시작	간선 없음, 노드마다 집합	노드 하나 u_0
매 단계	cycle을 안 만드는 가장 가벼운 간선	S–$V \setminus S$ 사이 가장 가벼운 간선
중간 X의 모양	여러 tree (forest)	하나의 tree
필요한 것	간선 정렬, union-find	priority queue, cost/prev, decreasekey
시간	$O(\|E\| \log \|V\|)$	$O(\|E\| \log \|V\|)$ (binary heap)
cut property의 S	새 간선이 잇는 두 tree 중 하나	지금까지 만든 subtree

Union-Find (disjoint sets)

연산과 자료구조

- makeset(x): x만 든 새 집합 · find(x): x가 속한 집합의 root 노드 (집합의 이름) · union(x, y): 두 집합을 합침
- $\pi(x)$: x의 parent 포인터 (root는 자기 자신) · $\mathrm{rank}(x)$: x를 root로 하는 부분트리의 높이 (슬라이드 정의). 정확히는 union by rank가 유지하는 저장값 — path compression으로 실제 높이가 줄어도 갱신하지 않고, parent를 따라가면 rank는 엄격히 증가한다 (아래 분할 상환 분석이 쓰는 성질)
- **union by rank**: rank가 낮은 root를 높은 root 아래에 붙임 → tree를 낮게 유지. rank가 같으면 아무쪽이나 (슬라이드는 $\pi(r_x) \leftarrow r_y$), 새 root의 rank +1. rank가 다르면 높이 그대로

손계산 4: union by rank 추적 (칸 = π/rank)

단계	연산	A	B	C	D	E	F	G
0	makeset × 7	A/0	B/0	C/0	D/0	E/0	F/0	G/0
1	union(A, D)	D/0	B/0	C/0	D/1	E/0	F/0	G/0
2	union(B, E)	D/0	E/0	C/0	D/1	E/1	F/0	G/0
3	union(C, F)	D/0	E/0	F/0	D/1	E/1	F/1	G/0
4	union(C, G)	D/0	E/0	F/0	D/1	E/1	F/1	F/0
5	union(E, A)	D/0	E/0	F/0	D/2	D/1	F/1	F/0
6	union(B, G)	D/0	E/0	F/0	D/2	D/1	D/1	F/0

- 최종: root D (rank 2), 노드 7개, 높이 2 · 연결 성분 수 7 → 4 (union 3번) → 2 (union 2번) → 1 (union 1번)
- 높이 $\log n$은 실제로 나옴 — 같은 rank끼리만 합치면 $n = 2^k$에서 rank k
- Kruskal 알고리즘 자체는 단순, union-find가 더 중요한 자료구조 (cycle 검사 말고 다른 용도에도 쓰임)

rank의 성질

① $\mathrm{rank}(x) \le \mathrm{rank}(\pi(x))$ — root로 올라갈수록 rank는 증가

② rank는 root일 때만 바뀌고, non-root가 다시 root가 되는 일은 없음

③ rank k인 노드는 rank가 그 값으로 정해진 시점(그때는 root)의 부분트리에 노드를 **≥ 2^k개** 가짐

④ 귀납: rank k는 rank $k-1$ 둘의 합성 → $\ge 2^{k-1} + 2^{k-1} = 2^k$

⑤ 승격 시점 기준 주장: path compression으로 자식이 떨어져 나가도 rank는 갱신되지 않으므로 현재 부분트리는 이보다 작을 수 있다

⑥ rank k인 노드는 **≤ $n/2^k$개**

⑦ 같은 rank k를 가진 노드들의 승격 시점 부분트리는 서로 겹치지 않으므로 — 각 노드는 최대 한 rank-k 노드의 그 시점 부분트리에 속함

⑧ → $\mathrm{rank} \le \log n$, 높이 $\le \log n$ → find, union $O(\log n)$

$n = 16$ 예: k = 0, 1, 2, 3, 4일 때 최대 개수 16 / 8 / 4 / 2 / 1

Path Compression (경로 압축)

find(x)가 지나가는 모든 노드를 root에 직접 연결해 tree를 더 낮게 유지한다. 슬라이드 코드:

```
find(x):
  if x ≠ π(x) then π(x) ← find(π(x))
  return π(x)
```

root를 찾는 순회와 압축 순회, 두 번 지나간다 (one-pass 변형으로 path halving: `π(x) ← π(π(x))`를 따라가며 건너뛰기 — find(x)는 x ≠ π(x)인 동안 π(x) ← π(π(x))로 건너뛰고 x ← π(x)로 전진하는 반복이며, 루트에 닿으면 그 루트를 반환한다).

분할 상환(amortized) 분석 (슬라이드 15-17) — 목표 → 가정과 정의 → 핵심 관찰 → 수식 전개 → 결론 순으로 읽는다.

① **목표**:

union by rank만으로는 find가 `O(log n)`이지만(위 rank의 성질 4), path compression을 더하면 find m번(m은 find 호출 횟수)과 union n−1번을 임의 순서로 실행해도 전체 시간이 `O(m log* n)`에 묶임(분할

상환)을 보인다.

개별 find의 비용을 find 자신과 경로 위 노드들에게 나눠 청구하는 예산 방식으로 증명한다.

② **가정과 정의**:

분석이 쓰는 성질은 rank의 성질 1 — parent를 따라가면 rank가 엄격히 증가한다.

먼저 **rank 그룹 나누기**: rank의 범위 0 ≤ rank ≤ log n을 구간 $\{k+1, \cdots, 2^k\}$ (슬라이드 15의 그룹 나열 {1}, {2}, {3, 4}, {5, ⋯, 16}, {17, ⋯, 2^{16}}, ⋯ 와 동일;

경계 k = 0, 1, 2, 4, 16, 65536, ⋯, 다음 경계 = 2^(앞 경계))로 묶으면 실제로 나타나는 양의 rank 그룹 수는 $\log^* n$ 이하

($\log^* n$ = n에 밑이 2인 로그 $\log_2$를 1 이하가 될 때까지 반복 적용한 횟수; rank가 최대 log n이어도 작은 n에서는 그룹이 다 차지 않을 수 있으므로 등식이 아니라 상한).

예: $\log^* 65536 = 4$ (65536 → 16 → 4 → 2 → 1).

③ **핵심 관찰**: find 경로 위 노드 x는 둘로 나뉜다:

④ **A** π(x)와 다른 그룹 — 최대 log* n개, find가 지불

⑤ **B** 같은 그룹 — x가 지불한다

⑥ **수식 전개**:

각 find에는 $\log^* n + O(1)$ 달러, rank가 구간 $\{k+1..2^k\}$인 노드 x에는 2^k 달러를 준다 (한 구간의 노드 수 ≤ $n/2^k$라 노드에게 주는 총액 ≤ n log* n).

단 root가 아닌 노드 중 π(x)가 이미 root인 노드는 경로당 최대 하나뿐이라 (root 자체의 종료 검사와 함께) find의 주머니에서 O(1)로 처리하고,

그 밖의 x는 지불할 때마다 π(x)가 더 rank가 높은 root로 바뀌므로 π(x)가 그룹을 벗어나기 전 최대 2^k번만 지불하고 이후엔 영구히 A가 된다.

(rank 0인 비-root 노드는 어느 그룹에도 속하지 않지만(그룹은 $\{k+1,\ldots,2^k\}$), parent의 rank가 1 이상이라 한 find 경로에 최대 하나뿐이라 별도로 O(1) 처리할 수 있음.

이처럼 root 자신과 rank 0 주변의 경계 예외는 find마다 O(1)로 처리하므로, 양의 rank 그룹 수 상한과 합쳐 find당 예산 log* n + O(1)이 유지된다.)

⑦ **결론**:

find m번 + union n−1번의 임의 순서가 $O(m \log^* n + n \log^* n) = O(m \log^* n)$ (m > n).

더 정밀한 한계는 Tarjan(1975)의 $\Theta(m \cdot \alpha(m,n))$ — α는 Ackerman 함수의 역함수로, 슬라이드가 쓰는 정의 기준 $n < 2^{65536}$에서 $\alpha \leq 3$이라 사실상 상수.

즉 정렬을 빼면 Kruskal의 union-find 부분은 거의 선형이다.

CSED331 · Greedy · Union-Find

단계 1 / 10 · makeset × 7

Union-Find 단계별 시뮬레이션

union by rank + path compression

이전 | 다음 | 초기화 | 자동 재생

Parent 트리



Parent 포인터: 자식 → 부모 find(G) 호출 전 경로

경로 압축 후 포인터 원 안: 정점 / rank

현재 단계의 상태

현재 연산

makeset(A), makeset(B), makeset(C), makeset(D), makeset(E), makeset(F), makeset(G)

Root 수

7

연결 성분 수

7

Root 목록

A {A} | **B** {B} | **C** {C} | **D** {D} | **E** {E} | **F** {F} | **G** {G}

단계 설명

A, B, C, D, E, F, G를 각각 독립 집합으로 초기화한다. 모든 정점은 자기 자신을 부모로 가지며 rank=0이다.

현재 연산의 결과

각 정점 x에 대해 π(x)=x, rank(x)=0이다. 연결 성분 수는 7.

π(x) / rank 표

정점 x	π(x)	rank(x)
A	A (root)	0
B	B (root)	0
C	C (root)	0
D	D (root)	0
E	E (root)	0
F	F (root)	0
G	G (root)	0

π(x)는 parent 포인터다. Root는 π(x)=x이다. rank(x)는 x를 root로 하는 부분트리의 높이(슬라이드 정의)에 기반한 저장값이다.

연산 규칙

makeset(x): x만 든 새 집합을 만든다.

find(x): x가 속한 집합의 root 노드를 반환한다.

union(x, y): 두 집합을 합친다.

union by rank: rank가 낮은 root를 높은 root 아래에 붙인다. rank가 같으면 슬라이드는 첫 번째 인자의 root를 두 번째 root 아래에 붙이고 새 root의 rank를 1 증가시킨다. rank가 다르면 rank는 그대로다.

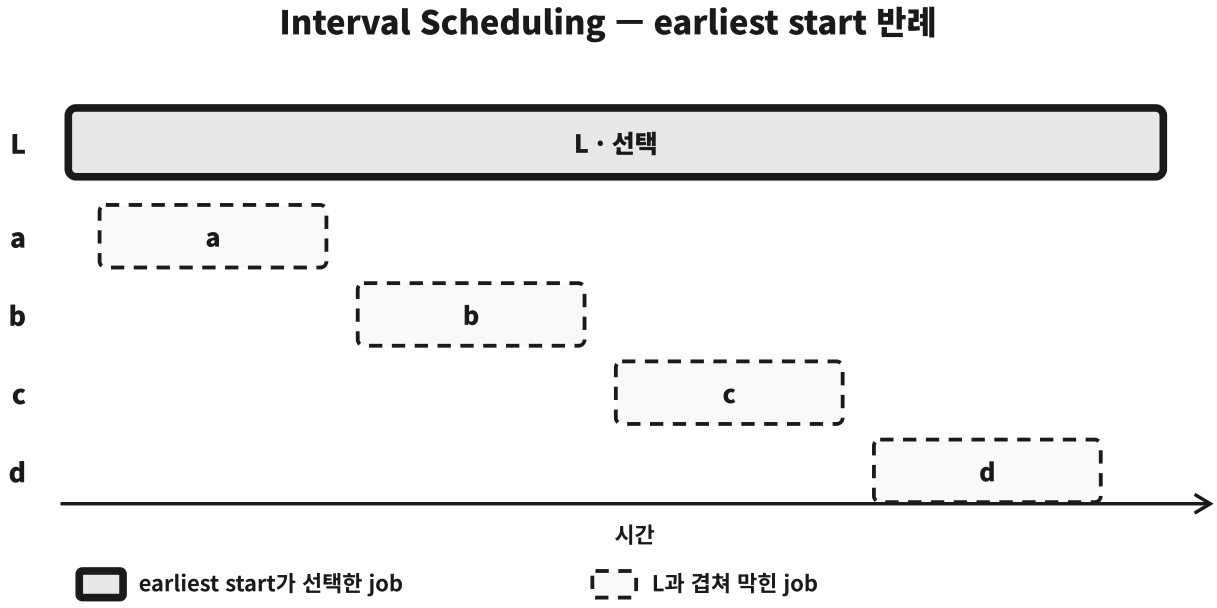
Path compression: find 중 지나간 노드의 parent를 root로 직접 연결한다. 실제 트리 높이가 줄어도 rank 저장값은 갱신하지 않는다.

```
find(x):
  if x ≠ π(x) then
    π(x) ← find(π(x))
  return π(x)
```

Interval Scheduling

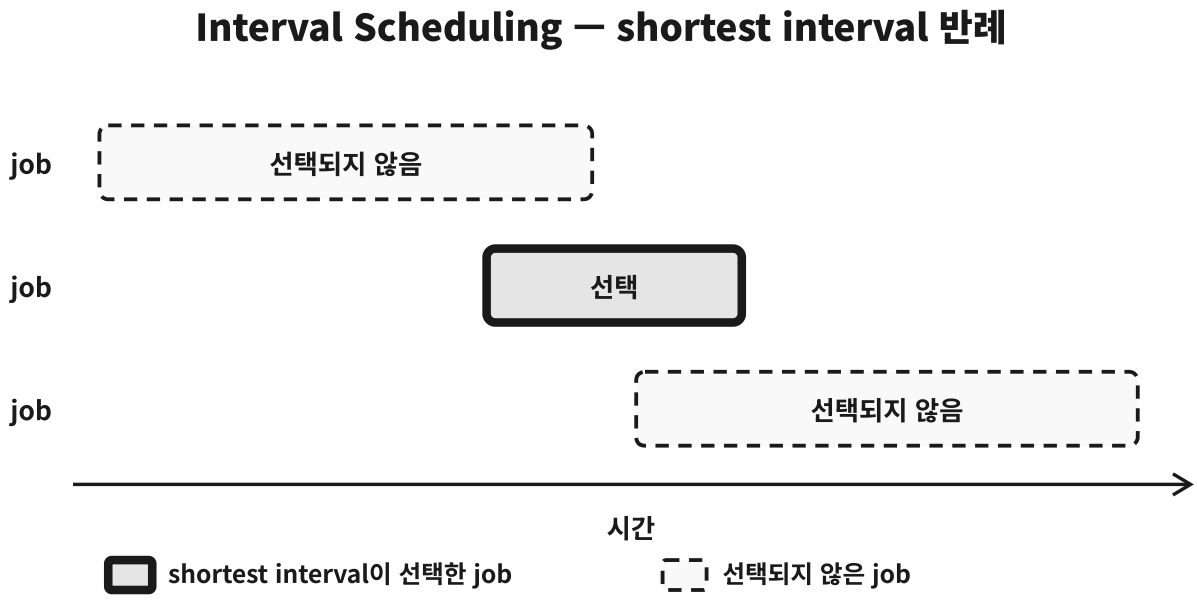
문제와 greedy 후보

- job j가 s_j에 시작해 f_j에 끝남 (반열린 구간 $[s, f)$). 서로 안 겹치면 **compatible**. 목표: 서로 compatible한 job 부분집합의 **최대 크기**
- 후보 규칙 4개: earliest start time (s_j 오름차순) · earliest finish time (f_j 오름차순) · shortest interval ($f_j - s_j$ 오름차순) · fewest conflicts (충돌 수 c_j 오름차순)



earliest start: 1개 vs 최적 4개

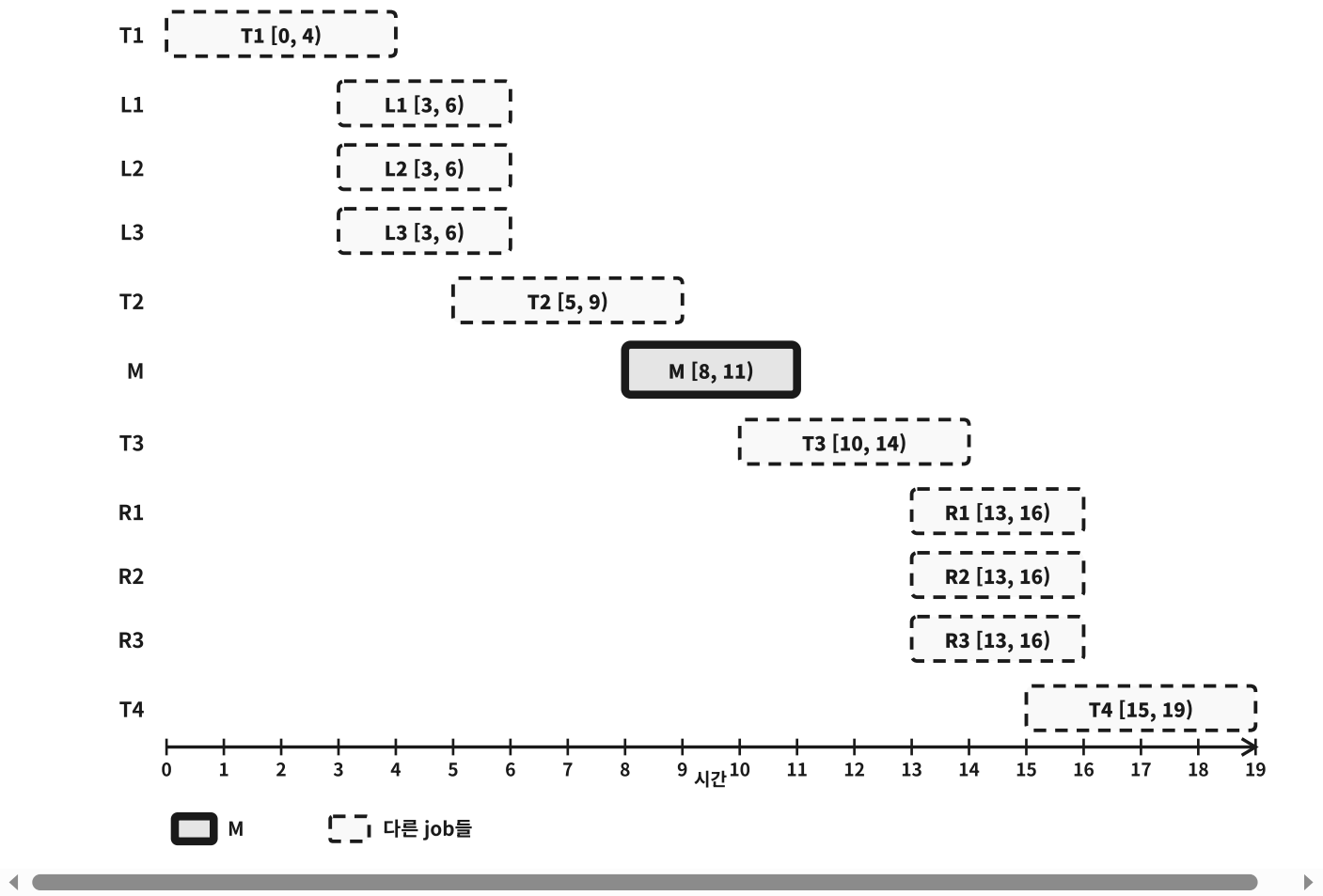
L 하나를 선택하면 1개 · a, b, c, d를 선택하면 4개



shortest interval: 1개 vs 최적 2개

가운데 job 하나 선택 vs 양쪽 job 두 개 선택

Interval Scheduling — fewest conflicts 반례



충돌 2회인 M이 충돌 4회인 T2, T3보다 적어 보이지만
M을 고르면 T2, T3를 못 고름 → 최적 4개 vs 3개

earliest finish time = 네 가지 후보 규칙 중 항상 최적해를 보장하는 규칙

반례 셋 — earliest start: 먼저 시작하는 긴 구간 L 하나가 짧은 4개를 전부 막음 (1개 vs 최적 4개) · shortest interval: 가운데 짧은 구간 M이 양옆 긴 구간 A, B와 모두 겹침 (1개 vs 최적 2개) · fewest conflicts: 충돌이 가장 적은 M이 최적 해 밖, M을 고르면 T2, T3를 못 고름 (3개 vs 최적 4개)

- fewest conflicts 반례의 구간과 충돌 수 (슬라이드 그림과 같은 모양, 값은 직접 정함): M [8, 11) $c_j = 2$ · T1, T4 [0, 4), [15, 19) $c_j = 3$ · T2, T3 [5, 9), [10, 14) $c_j = 4$ · L1–L3 [3, 6) 셋, R1–R3 [13, 16) 셋 $c_j = 4$
- Earliest start 반례: L = [0, 10), 짧은 구간 [1, 2), [3, 4), [5, 6), [7, 8). L을 먼저 선택하면 1개, 짧은 구간 4개는 서로 양립해 최적 4. Shortest interval 반례: A = [0, 5), M = [4, 6), B = [5, 10). M은 A·B 모두와 겹쳐 1개, A와 B는 반열린 구간이라 양립해 최적 2.

earliest finish time: 종료 시각 f_j 오름차순으로 보며, 마지막으로 고른 job과 compatible ($s_j \geq$ 마지막 종료) 하면 선택. 시간 $O(n \log n)$.

비교한 네 가지 후보 규칙 중 earliest finish time은 항상 최적해를 보장한다. 다른 그리디 전략도 구조에 따라 최적일 수 있어 최적성은 별도 증명이 필요하다.

손계산 5: earliest finish time

종료 순	job	s	f	판정	마지막 선택 f
1	1	1	3	**선택**	3
2	2	2	5	충돌 (시작 2 < 3)	3
3	3	4	7	**선택**	7
4	4	1	8	충돌 (시작 1 < 7)	7
5	5	6	9	충돌 (시작 6 < 7)	7
6	6	8	10	**선택**	10
7	7	9	12	충돌 (시작 9 < 10)	10
8	8	11	14	**선택**	14

최적성 증명: 교환 논증

① greedy가 고른 job $i_1, i_2, \ldots, i_k$. 최적해 중 $i_1 = j_1, \ldots, i_r = j_r$이 되는 r이 **가장 큰** 것을 잡음: $j_1, \ldots, j_m$

② r번째까지 같으므로 j_{r+1}을 greedy의 i_{r+1}로 바꿔도 feasible — i_{r+1}은 $j_r = i_r$ 이후 끝나는 job 중 종료가 가장 빠르므로 $f_{i_{r+1}} \leq f_{j_{r+1}}$, 그 뒤 j_{r+2}와도 안 겹침

③ 그러면 $r+1$까지 일치하는 최적해가 존재 → r의 최대성에 모순 → greedy = 최적 ∎

Interval Partitioning

- 강의 n개, 강의 j는 $[s_j, f_j)$. 같은 강의실에서 시간이 겹치지 않게 **모든** 강의를 배정 → 목표: **강의실 수 최소**
- **depth** = 어떤 시각에 동시에 겹치는 강의의 최대 수 → 필요한 강의실 수 $\geq$ depth (겹친 강의는 서로 다른 강의실 필요) = **하한**

① 시작 시각 순으로 정렬, $d = 0$

② 각 강의 j를 이미 연 강의실 중 compatible한 곳에 배정, 없으면 새 강의실 $d + 1$을 열어 배정

③ 구현: 강의실별 마지막 종료 시각을 최소 힙에 저장한다. 강의를 시작 시각 오름차순으로 처리한다.

힙의 최소 종료 시각이 현재 강의의 시작 시각 이하이면 해당 강의실을 재사용하고 종료 시각을 갱신한다. 최소 종료 시각이 시작 시각보다 크면 모든 강의실이 사용 중이므로 새 강의실을 연다.

강의 정렬과 힙 연산을 합친 시간복잡도는 $O(n \log n)$

손계산 6: interval partitioning

시작 순	강의	s	f	호환 강의실	배정	강의실 1, 2, 3의 마지막 종료
1	a	1	5	없음	1 (새로 엶)	5, –, –
2	b	2	4	없음	2 (새로 엶)	5, 4, –
3	c	3	7	없음	3 (새로 엶)	5, 4, 7
4	d	4	6	2	2	5, 6, 7
5	f	5	8	1	1	8, 6, 7
6	e	6	9	2	2	8, 9, 7
7	g	7	10	3	3	8, 9, 10
8	h	8	11	1	1	11, 9, 10

- 호환 강의실이 여럿이면 번호가 가장 작은 곳 (직접 정한 규칙)

- depth = 3 (시각 4에 a, c, d가 동시에 진행) → greedy가 연 강의실 3개 = depth → 최적. 새 강의실은 앞 3개 강의에서만 열림 → 그 뒤는 모두 기존 강의실에 들어감
- **최적성**: 강의실 $d+1$은 강의 j가 다른 d개 강의실과 모두 겹쳐서 열림. 시작 시각 순 정렬이라 겹친 강의들은 모두 s_j 이전/같은 시각에 시작 → 시각 $s_j+\epsilon$에 $d+1$개가 겹침 → depth $\geq d+1$ → **greedy의 강의실 수 = depth** = 하한 도달 → 최적

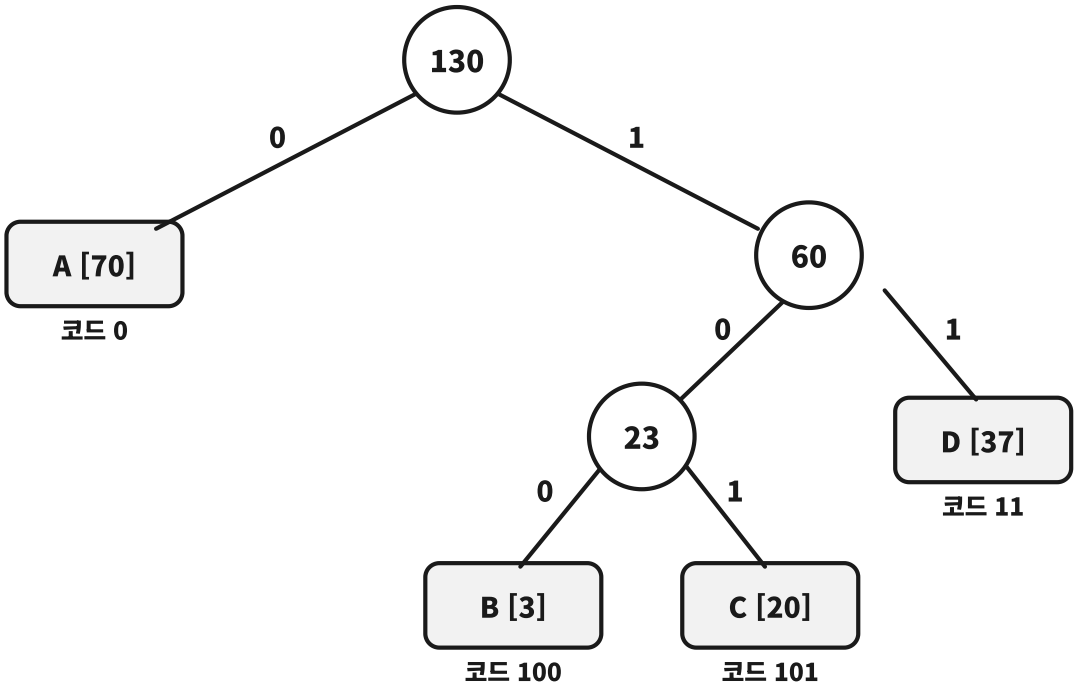
Huffman Encoding

고정 길이 vs 가변 길이

- 문자 1억 3천만 개, 알파벳 {A, B, C, D}: 고정 길이 $\{A=00, B=01, C=10, D=11\}$ → $130 \times 2 = 260$ Mbits
- 빈도에 따른 **variable-length encoding**을 쓸 때 즉시 복호화(앞에서부터 모호함 없이 바로 읽기)하려면 → **prefix-free** (어떤 코드 단어도 다른 코드 단어의 prefix가 아님 · 유일 복호화의 충분조건)
- 허프만 알고리즘은 매 단계 두 노드를 새 내부 노드의 자식으로 결합하므로 생성되는 코딩 트리는 full binary tree이다. Full binary tree란 모든 내부 노드가 정확히 두 자식을 갖는 이진 트리를 뜻한다. 이 성질은 허프만 알고리즘의 구성 방식에서 보장되지만, 허프만 트리에만 한정되는 성질은 아니다.

  각 leaf는 하나의 문자를 나타내며, root에서 leaf까지의 경로에서 왼쪽 간선은 0, 오른쪽 간선은 1로 표시하여 코드 단어를 만든다. Prefix-free는 어떤 코드 단어도 다른 코드 단어의 접두사가 되지 않는 성질이며, full binary tree 여부와는 구분해야 한다.

  예: A = 0, B = 100, C = 101, D = 11 → 고정 길이 260 대비 47 절약 (약 18%)



---

cost = 70·1 + 3·3 + 20·3 + 37·2 = 213
= 23 + 60 + (70+3+20+37) = 213

빈도 $f_1, \ldots, f_n$일 때 tree의 비용 — 두 가지 해석 (시험에 둘 다):

$$\text{cost of tree} = \sum_{i=1}^{n} f_i \cdot \text{depth}_i \;=\; \sum_{\text{nonroot internal } v} f(v) \;+\; \sum_{i=1}^{n} f_i$$

$f(v)$ = v의 부분트리 leaf 빈도 합 (인코딩/디코딩 때 v를 지나는 횟수). 위 예:
$70 \cdot 1 + 3 \cdot 3 + 20 \cdot 3 + 37 \cdot 2 = 213 = 23 + 60 + 130 = 213$

서로 다른 문자가 $n \geq 2$개인 full binary Huffman tree T의 비용은 다음 두 가지 방법으로 계산할 수 있다. $\text{cost}(T) = \sum_{i=1}^{n} f_i \cdot \text{depth}_T(i) = \sum_{v \text{ is a nonroot internal node}} f(v) + \sum_{i=1}^{n} f_i$. 여기서 $f(v)$는 v의 부분트리에 속한 leaf들의 빈도 합이다. 두 번째 등식은 $n \geq 2$일 때 루트가 내부 노드이며, 루트의 빈도가 전체 문자 빈도의 합 $f(\text{root}) = \sum_{i=1}^{n} f_i$라는 사실을 이용한다. 문자가 하나뿐인 경우에는 허프만 알고리즘의 병합이 일어나지 않고 루트 자체가 leaf가 되므로, 위 두 번째 등식은 적용하지 않는다. 단일 문자 입력을 실제로 인코딩해야 한다면 코드 0처럼 비어 있지 않은 코드워드를 부여하는 별도 규약을 명시하고, 그 규약에 따라 비용을 계산한다.

핵심 Lemma: 가장 낮은 두 빈도는 최하단 형제 (교환 증명)

① **가정과 정의**: 최적 tree T에서 최대 깊이 d의 형제 leaf를 a, b, 가장 낮은 빈도 글자를 x, y라 하자

② **수식 전개**: x와 a를 교환한 tree T': $\Delta = d - \text{depth}_T(x) \geq 0$,
$\text{cost}(T') = \text{cost}(T) + \Delta \cdot (f[x] - f[a])$

③ **결론**: $f[x] \leq f[a]$이므로 $\text{cost}(T') \leq \text{cost}(T) \rightarrow T'$도 최적. y와 b도 교환 → x, y가 최하단 형제인 최적 tree T'' 존재 ∎

알고리즘: priority queue H에 모든 글자와 그 빈도를 넣고, 하나가 남을 때까지 반복한다.

```
deletemin 두 번 → 빈도가 가장 낮은 두 노드 i, j를 꺼낸다
새 노드 k를 만든다: 자식은 i, j이고 빈도는 f[k] = f[i] + f[j]
k를 Q에 삽입한다
```

시간 $O(n \log n)$.

① 허프만 알고리즘의 최적성 증명 — 귀납법. 문자 수가 1개인 경우는 자명하다. 문자 수가 2개 이상이라고 하고, 가장 낮은 두 빈도를 $f_1 \leq f_2$라 하자. 앞의 Lemma에 따라 f_1, f_2를 형제 leaf로 갖는 최적 코딩 트리가 존재한다.

이 두 leaf와 그 부모를 하나의 leaf z로 축약하고, $f(z) = f_1 + f_2$로 정의하면 문자 수가 하나 줄어든 문제가 된다. 원래 트리의 비용은 축약된 트리의 비용에 $f_1 + f_2$를 더한 값이다. 두 문자를 다시 형제 leaf로 펼치면 각 문자의 깊이가 1씩 증가하기 때문이다.

축약된 트리가 축약 문제의 최적 트리가 아니라고 가정하자. 그러면 더 낮은 비용을 갖는 축약 트리로 바꾼 뒤, z를 빈도 f_1, f_2인 두 형제 leaf로 펼칠 수 있다. 이렇게 하면 원래 문제에서도 비용이 더 낮은 코딩 트리를 얻어 원래 트리의 최적성에 모순된다. 따라서 축약된 트리는 축약 문제의 최적 트리다.

귀납 가정에 따라 허프만 알고리즘은 축약된 문제를 최적으로 해결한다. 허프만 알고리즘의 첫 단계는 가장 낮은 두 빈도 f_1, f_2를 결합하는 것이므로, 축약 문제에서 얻은 최적 트리를 펼치면 원래 문제의 최적 트리를 얻는다. 따라서 귀납적으로 허프만 알고리즘은 최적의 prefix-free 코드를 생성한다.

손계산 7: Huffman (A 70, B 3, C 20, D 37)

단계	힙 (작은 순)	꺼낸 둘	새 노드	누적 cost
0	B 3, C 20, D 37, A 70			0
1	N1 23, D 37, A 70	B 3, C 20	N1 = 23	23

2	N2 60, A 70	N1 23, D 37	N2 = 60	83
3	N3 130	N2 60, A 70	N3 = 130	213

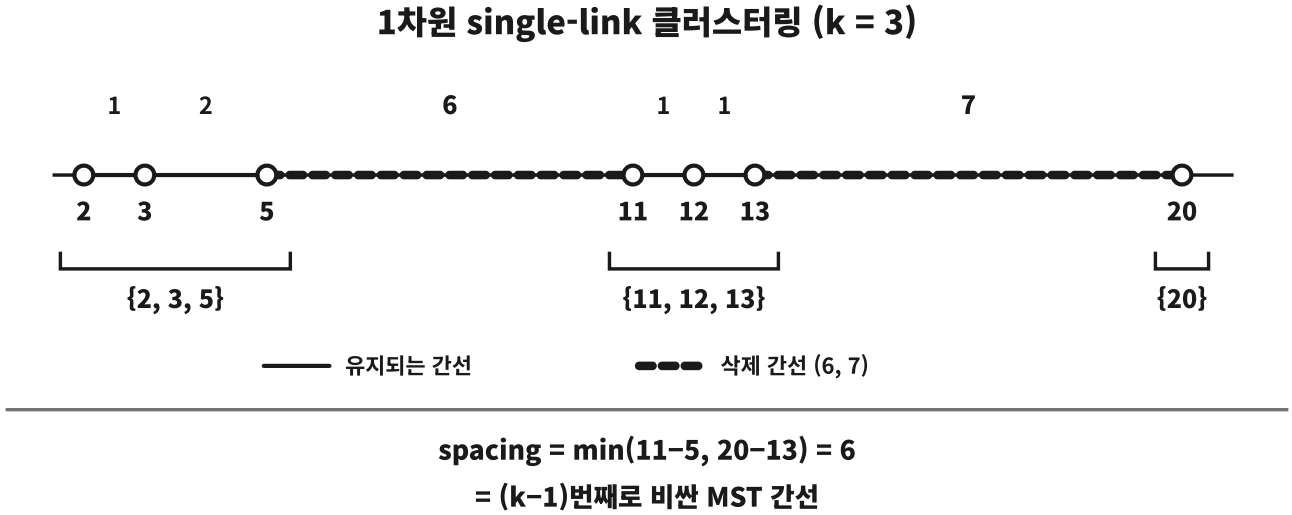
- ABDCA → 0 100 11 101 0 = 0100111010, 왼쪽부터 읽으면 0 = A, 100 = B, 11 = D, 101 = C, 0 = A로 유일하게 디코딩됨
- 병합 노드의 f 합 23 + 60 + 130 = 213 = 글자별 f × 길이의 합 — 두 노드를 합칠 때마다 새 노드의 f가 cost에 한 번씩 더해짐 (알고리즘의 근거)
- 코드 길이: 빈도가 큰 A가 가장 짧고 B, C가 가장 김 · 노드마다 0/1 배치를 바꿔도 코드 길이는 그대로 → 코드는 유일하지 않아도 cost는 같음

Clustering of Maximum Spacing

문제 정의

- n개 object를 비슷한 것끼리 k개의 nonempty group으로 분류 (**k-clustering**). 거리 함수는 두 object의 closeness를 반영
- **spacing** = 서로 다른 cluster에 속한 두 점 사이의 최소 거리. 목표: spacing이 **최대**인 k-clustering

> **single-link k-clustering**: 점 n개를 각각 cluster 하나로 시작 (정점 집합 U 위의 그래프) → 서로 다른 cluster에 있는 가장 가까운 두 점을 잇는 간선 추가 → $n-k$번 반복 → 정확히 k개 cluster.
> **= Kruskal을 연결 성분 k개에서 멈춘 것 = MST를 만든 뒤 가장 비싼 $k-1$개 간선을 지운 것**



예제 (연습문제 10 데이터 2, 3, 5, 11, 12, 13, 20). 손계산 8의 슬라이드 수치는 노트에 남아 있지 않아 이 데이터로 대체

정당성: single-link가 max spacing을 달성

① **가정과 정의**: single-link 결과 C의 spacing을 d라 하자. C의 각 cluster를 잇는 forest의 모든 간선 길이는 d 이하이다.

② **핵심 관찰**: 다른 k-clustering C*가 C와 다르면, C의 같은 cluster에 속하지만 C*의 서로 다른 cluster에 속하는 두 점이 존재한다.

③ **수식 전개**: 두 점을 잇는 C의 forest 경로에는 C*의 서로 다른 cluster를 잇는 간선 (p,q)가 있다. 이 간선 길이는 d 이하이다.

④ **결론**: C*의 spacing d*는 서로 다른 cluster 사이의 최소 거리이므로 $d^* \leq |pq| \leq d$. 따라서 C가 max spacing을 달성한다. ∎

슬라이드 정리 문장은 간선 "k개"로 적혀 있으나 **$k-1$개가 맞음** (바로 앞 슬라이드와 증명이 $k-1$ 기준).

연습문제

문제 1

$C = 400$, 충전소 위치 0, 120, 300, 380, 520, 700, 780, 1000 (도착 = 1000)일 때 SelectBreakpoints가 고르는 충전소와 충전 횟수는? 700, 780 두 곳이 없다면?

힌트 보기

매번 $x + C$ 안에서 **가장 먼** 충전소 b_p를 고름. $b_p = x$이면 no solution.

정답 보기

개념은 위 힌트 참고.

① $x = 0$, $x + C = 400$ → 가장 먼 역은 380 (520은 400 초과)

② $x = 380$, $x + C = 780$ → 가장 먼 역은 780

③ $x = 780$, $x + C = 1180$ → 1000 도착 가능, 종료

④ 1번만 충전하면 최대 780 < 1000 → 3개 구간 필요 → 2번이 최소

⑤ 700, 780이 없으면 0 → 380 → 520. 520에서는 920까지밖에 못 가고 다음 역이 자기 자신 → $b_p = x$ → no solution

최종 답 **충전소 380, 780 — 충전 2번 · 700/780이 없으면 no solution**

문제 2

슬라이드 3 그래프 (A–C 1, C–D 2, A–D 3, A–B 4, B–C 4, B–D 4, C–F 4, E–F 5, D–F 6)에서 Kruskal을 돌려 간선 판정 순서와 MST 합을 구하고, MST가 몇 개인지 답하시오.

힌트 보기

가중치 순으로 보며 find(u) ≠ find(v)면 추가. 가중치 4짜리 간선이 4개 → MST 개수는 동점 선택지의 경우의 수.

정답 보기

개념은 위 힌트 참고.

① A–C 1 추가, C–D 2 추가, A–D 3 버림 (cycle A–C–D)

② 4짜리 중 먼저 본 것: B–D 추가, C–F 추가 (순서에 따라 A–B나 B–C가 먼저 올 수도 있음), A–B·B–C는 그 뒤라 버림

③ E–F 5 추가 (E의 유일한 연결), D–F 6 버림 → 간선 5개로 종료

④ 공통 4간선 (A–C, C–D, C–F, E–F = 12) + B를 잇는 간선 하나 (A–B, B–C, B–D 중 아무거나 4) — 셋 다 cycle 없이 B를 연결

최종 답 **합 16 — MST는 3개 (B를 잇는 간선 선택에 따라)**

문제 3

슬라이드 5 그래프에서 Prim을 노드 E에서 시작해 뽑는 순서와 MST 합을 구하시오. 동점이 생기는 단계가 있는가?

힌트 보기

cost(z) = S와 z를 잇는 최소 간선. deletemin으로 최소를 뽑고 이웃 cost/prev를 갱신.

정답 보기

개념은 위 힌트 참고.

S	A	B	C	D	E	F
	∞/nil	∞/nil	∞/nil	∞/nil	0/nil	∞/nil
E	∞/nil	∞/nil	5/E	∞/nil		4/E
E, F	∞/nil	∞/nil	3/F	4/F		
E, F, C	6/C	1/C		2/C		
E, F, C, B	5/B			2/C		
E, F, C, B, D	4/D					

① 매 단계 표에서 최소 cost를 뽑음: E → F(4) → C(3) → B(1) → D(2) → A(4)

② 간선 E–F, F–C, C–B, C–D, D–A → Kruskal 결과와 같은 tree (C–D를 쓴 쪽)

③ 각 단계의 최솟값이 하나뿐 → 동점 없음

최종 답 **순서 E → F → C → B → D → A, 합 14 — 동점 단계 없음**

문제 4

① (a) $\log^* 65536$을 구하시오

② (b) $n = 1000$일 때 최대 rank, rank 5인 root의 최소 노드 수, rank 5인 노드의 최대 개수는?

③ (c) rank 9까지 필요한 그룹 수를 $\log^* 1000$과 비교하시오.

힌트 보기

정의	$\log^* n$ = 1이 될 때까지 log를 취한 횟수
rank 성질	rank k root는 $\geq 2^k$ 노드, rank k 노드는 $\leq n/2^k$개
그룹	{1}, {2}, {3, 4}, {5, ⋯, 16}, {17, ⋯, 65536}, ⋯ · 다음 경계 = 앞 경계만큼 2를 거듭제곱 (1 → 2 → 4 → 16 → 65536)

정답 보기

개념은 위 힌트 참고.

① (a) 65536 → 16 → 4 → 2 → 1 : log 4번 → $\log^* 65536 = 4$

② (b) 최대 rank $\leq \lfloor \log_2 1000 \rfloor = 9$ · rank 5 root $\geq 2^5 = 32$ 노드 · rank 5 노드 $\leq \lfloor 1000/32 \rfloor = 31$개

③ (c) 그룹 {1}, {2}, {3, 4}, {5, ⋯, 16} = 4개 (rank 9 ∈ {5..16}) · $\log^* 1000 = 4$ (1000 → 10 → 4 → 2 → 1⋯ 아니라 1000 → $\log_2$ 9.97 → 3.32 → 1.73 → 0.79: 4번) → 그룹 수와 일치

최종 답 **(a) 4 · (b) 9 / 32 / 31 · (c) 그룹 4개 =** $\log^* 1000 = 4$

문제 5

슬라이드 5 그래프에서 B–D의 가중치를 2에서 3으로 바꾸면 Kruskal 판정 순서, MST 합, MST 개수는? B–D가 MST에 못 들어가는 이유는?

힌트 보기

사이클에서 **유일하게** 가장 무거운 간선은 어떤 MST에도 포함되지 않는다. 이를 cycle property라 한다. cut property와 관련 있으며 교환 논증으로 증명할 수 있다.

정답 보기

개념은 위 힌트 참고.

① B–C 1 추가, C–D 2 추가 (이제 B, C, D 같은 집합)

② B–D 3 버림 (같은 집합), C–F 3 추가, A–D 4 추가, D–F 4 버림, E–F 4 추가 → 간선 5개 완성

③ B–D가 2일 때는 C–D 2와 동점이라 MST가 2개였으나, 3이 되면 C–D 2가 확실히 더 가벼움 → 선택이 하나로 정해짐

④ cycle B–C–D (가중치 1, 2, 3)에서 B–D 3이 가장 무거움 → 어떤 MST에도 못 들어감

최종 답 **합 14, MST 1개 — B–D 3은 cycle B–C–D의 최중량 간선이라 영구 탈락**

문제 6

job 7개 $[s, f)$: 1 [4, 10), 2 [1, 4), 3 [0, 5), 4 [14, 21), 5 [13, 15), 6 [10, 14), 7 [8, 16).

① earliest finish time greedy가 보는 순서와 고르는 job을 구하시오

② earliest start, shortest interval로 골랐을 때와 크기를 비교하시오

힌트 보기

각 규칙의 정렬 기준대로 순서를 세우고, 마지막 선택 job의 종료 이후에 시작하는 첫 job을 선택.

정답 보기

개념은 위 힌트 참고.

① 종료 순: 2 (4), 3 (5), 1 (10), 6 (14), 5 (15), 7 (16), 4 (21)

② 2 [1, 4) 선택 → 3 [0, 5) 충돌 → 1 [4, 10) 선택 → 6 [10, 14) 선택 → 5 [13, 15) 충돌 → 7 [8, 16) 충돌 → 4 [14, 21) 선택

③ earliest start (시작 순 3, 2, 1, 7, 6, 5, 4): {3, 7} = 2개 · shortest interval (길이 순 5, 2, 6, 3, 1, 4, 7): {5, 2, 1} = 3개

최종 답

earliest finish: {2, 1, 6, 4} = 4개 (최적, 완전탐색 최댓값도 4) — 나머지 규칙은 2개, 3개로 비최적

문제 7

강의 7개 $[s, f)$: p1 [0, 3), p2 [1, 5), p3 [2, 4), p4 [3, 6), p5 [4, 8), p6 [5, 7), p7 [6, 9).

① depth를 구하시오

② interval partitioning greedy의 배정(번호가 작은 호환 강의실 우선)과 강의실 수를 구하시오

힌트 보기

depth = 한 시각에 겹치는 최대 강의 수. 시작 순으로 보며 호환되는 강의실이 있으면 배정, 없으면 새로 엶.

정답 보기

개념은 위 힌트 참고.

① depth = 3 (시각 2.5에 p1, p2, p3 겹침) → 강의실 3개 이상 필요

② p1 → 1 (새로 엶), p2 → 2 (새로), p3 → 3 (새로)

③ p4 → 1 (종료 3 ≤ 시작 3), p5 → 3 (종료 4 ≤ 시작 4; 강의실 2는 종료 5 > 4라 불가), p6 → 2 (종료 5 ≤ 시작 5), p7 → 1 (종료 6 ≤ 시작 6)

최종 답

배정 p1→1, p2→2, p3→3, p4→1, p5→3, p6→2, p7→1 — 강의실 3개 = depth → 최적

문제 8

빈도 a 12, b 3, c 25, d 8, e 40으로 Huffman 병합 순서, 코드 길이, cost를 구하고, 글자당 3비트 고정 길이와 비교하시오.

힌트 보기

가장 낮은 두 빈도를 반복해 합침. cost $= \sum f_i \cdot \mathrm{depth}_i$ = 병합 노드 값의 총합.

정답 보기

개념은 위 힌트 참고.

① 병합: 3 + 8 = 11 (b, d) → 11 + 12 = 23 (a와 합침) → 23 + 25 = 48 (c와 합침) → 40 + 48 = 88 (e와 합침)

② 깊이: e 1, c 2, a 3, b 4, d 4 → cost = 40·1 + 25·2 + 12·3 + 3·4 + 8·4 = 40 + 50 + 36 + 12 + 32

③ 검산: 병합 노드 합 11 + 23 + 48 + 88 = 170 일치

④ 고정 길이 3비트: 88 × 3 = 264

최종 답 **cost 170 — 고정 길이 264 대비 94 절약 (약 36%)**

문제 9

(a) 코드 집합 {0, 10, 110, 111}과 {0, 01, 11}이 각각 prefix-free인지 판단하시오. (b) 빈도가 모두 1인 글자 4개, 5개에 Huffman을 돌리면 병합과 cost는?

힌트 보기

prefix-free = 어떤 코드도 다른 코드의 앞부분이 아님. 빈도가 같으면 균형 잡힌 tree가 만들어짐.

정답 보기

개념은 위 힌트 참고.

① (a) {0, 10, 110, 111}: 어떤 코드도 다른 코드의 prefix가 아님 → prefix-free. {0, 01, 11}: 0이 01의 prefix → 0을 읽는 순간 끝인지 01의 시작인지 바로 정할 수 없음 → 아님

② (b) 4개: 병합 1+1=2, 1+1=2, 2+2=4 → 모두 깊이 2 → cost 8 (고정 길이 2비트와 같음)

③ 5개: 병합 1+1=2, 1+1=2, 2+1=3, 2+3=5 → 깊이 2, 2, 2, 3, 3 → cost 12 (고정 길이 3비트면 15)

최종 답 **(a) {0, 10, 110, 111} = prefix-free, {0, 01, 11} = 아님 · (b) 4개 → 8, 5개 → 12**

문제 10

1차원 점 2, 3, 5, 11, 12, 13, 20을 single-link로 $k = 3$, $k = 2$ cluster로 나누고 spacing을 구하시오. MST 간선도 함께 쓰시오.

힌트 보기

single-link = MST에서 가장 비싼 $k - 1$개 간선 삭제. spacing = 삭제한 간선 중 가장 가벼운 것의 길이. 단, 이 spacing 공식은 2 ≤ k ≤ n일 때의 것이다. k = 1이면 삭제할 간선이 없고 서로 다른 cluster 쌍도 없어 spacing이 정의되지 않는다.

정답 보기

개념은 위 힌트 참고.

① 인접 거리 1, 2, 6, 1, 1, 7 → MST 간선: 2–3 (1), 11–12 (1), 12–13 (1), 3–5 (2), 5–11 (6), 13–20 (7)

② $k = 3$: 가장 비싼 2개 (7, 6) 삭제 → {2, 3, 5}, {11, 12, 13}, {20}. spacing = min(11 − 5, 20 − 13) = 6

③ $k = 2$: 가장 비싼 1개 (7) 삭제 → {2, 3, 5, 11, 12, 13}, {20}. spacing = 7

최종 답 **k=3: {2,3,5} {11,12,13} {20}, spacing 6 · k=2: {2,3,5,11,12,13} {20}, spacing 7**